

Guia de Comandos SQL — Banco Lattes

Consultas para explorar pesquisadores, atividades, participações e coautorias no banco de dados do projeto de ETL do Lattes.

Objetivo: servir como uma referência rápida para entender o que cada consulta faz, por que ela é útil e quais partes do SQL são responsáveis pelo resultado.

Estrutura do banco utilizada

As consultas trabalham principalmente com quatro tabelas: PESQUISADORES, que guarda as pessoas; ATIVIDADES, que guarda as produções e atividades; PARTICIPACAO, que relaciona pesquisadores às atividades; e VINCULOS, usada para projetos, orientações e bancas. A tabela PARTICIPACAO é especialmente importante porque permite reconstruir as relações de autoria entre pesquisadores.

Fonte: comandos SQL presentes no arquivo fornecido pelo usuário. As explicações abaixo são uma interpretação didática desses comandos.

1. Contar todas as atividades

Por que usar: Saber quantos registros existem na tabela ATIVIDADES. É uma consulta básica de validação do ETL.

Comando:

```
SELECT COUNT(*) AS total_atividades
FROM ATIVIDADES;
```

Como funciona: COUNT(*) conta todas as linhas. O alias total_atividades dá um nome mais claro à coluna do resultado. É útil para conferir se a quantidade carregada no banco corresponde ao esperado.

2. Contar todos os pesquisadores

Por que usar: Saber quantas pessoas estão cadastradas no banco.

Comando:

```
SELECT COUNT(*) AS total_pesquisadores
FROM PESQUISADORES;
```

Como funciona: Assim como na consulta anterior, COUNT(*) conta as linhas. É útil para verificar quantos pesquisadores foram efetivamente cadastrados após o ETL.

3. Encontrar pares de pesquisadores e contar suas colaborações

Por que usar: Descobrir quais pesquisadores já participaram das mesmas atividades e quantas vezes isso ocorreu, inclusive separando a quantidade por tipo de produção.

Comando:

```
SELECT
  p1.nome AS pesquisador_1,
  p2.nome AS pesquisador_2,

  COUNT(*) FILTER (WHERE a.tipo = 'ARTIGO') AS artigos,
  COUNT(*) FILTER (WHERE a.tipo = 'LIVRO') AS livros,
  COUNT(*) FILTER (WHERE a.tipo = 'CONGRESSO_COMPLETO') AS congressos_completos,
  COUNT(*) FILTER (WHERE a.tipo = 'CONGRESSO_RESUMO') AS congressos_resumo,
  COUNT(*) FILTER (WHERE a.tipo = 'CONGRESSO_RESUMO_EXP') AS congressos_resumo_exp,
  COUNT(*) FILTER (WHERE a.tipo = 'APRESENTACAO') AS apresentacoes,
  COUNT(*) FILTER (WHERE a.tipo = 'OUTRA_PRODUCAO') AS outras_producoes,

  COUNT(*) AS total_atividades
```

```

FROM PARTICIPACAO pa1

JOIN PARTICIPACAO pa2
  ON pa1.id_atividade = pa2.id_atividade
  AND pa1.id_pesquisador < pa2.id_pesquisador

JOIN ATIVIDADES a
  ON a.id_atividade = pa1.id_atividade

JOIN PESQUISADORES p1
  ON p1.id_pesquisador = pa1.id_pesquisador

JOIN PESQUISADORES p2
  ON p2.id_pesquisador = pa2.id_pesquisador

GROUP BY
  p1.id_pesquisador,
  p1.nome,
  p2.id_pesquisador,
  p2.nome

ORDER BY total_atividades DESC;

```

Como funciona: O ponto central é o JOIN de PARTICIPACAO consigo mesma. Quando dois pesquisadores possuem o mesmo id_atividade, eles participaram da mesma produção. A condição pa1.id_pesquisador < pa2.id_pesquisador evita contar o mesmo par duas vezes (A-B e B-A). Os FILTER separam os resultados por tipo de atividade. O resultado pode ser usado como base para uma rede de coautoria.

4. Quantidade de atividades por ano

Por que usar: Ver como a produção cadastrada se distribui ao longo do tempo.

Comando:

```

SELECT
  ano,
  COUNT(*) AS quantidade
FROM ATIVIDADES
GROUP BY ano
ORDER BY ano;

```

Como funciona: GROUP BY ano cria um grupo para cada ano. Depois, COUNT(*) informa quantas atividades existem em cada grupo. ORDER BY ano coloca os anos em ordem crescente. É uma consulta útil para construir séries temporais.

5. Quantidade de atividades por tipo

Por que usar: Descobrir quais tipos de produção aparecem mais no banco.

Comando:

```

SELECT
  tipo,
  COUNT(*) AS quantidade
FROM ATIVIDADES
GROUP BY tipo
ORDER BY quantidade DESC;

```

Como funciona: Agrupa as atividades por tipo e conta quantas existem em cada categoria. A ordenação decrescente mostra primeiro os tipos com maior quantidade.

6. Número de colaboradores de cada pesquisador

Por que usar: Medir quantas pessoas diferentes já colaboraram com cada pesquisador.

Comando:

```

SELECT
  p.nome,
  COUNT(DISTINCT
    CASE

```

```

        WHEN pa1.id_pesquisador = p.id_pesquisador
        THEN pa2.id_pesquisador
        ELSE pa1.id_pesquisador
    END
) AS numero_colaboradores
FROM PESQUISADORES p
JOIN PARTICIPACAO pa1
    ON pa1.id_pesquisador = p.id_pesquisador
JOIN PARTICIPACAO pa2
    ON pa1.id_atividade = pa2.id_atividade
    AND pa1.id_pesquisador <> pa2.id_pesquisador
GROUP BY
    p.id_pesquisador,
    p.nome
ORDER BY numero_colaboradores DESC;

```

Como funciona: A consulta procura outras pessoas que aparecem nas mesmas atividades. COUNT(DISTINCT ...) é importante porque um mesmo colaborador pode aparecer em várias atividades, mas deve ser contado apenas uma vez para o número de colaboradores distintos.

7. Atividades por pesquisador e por tipo

Por que usar: Ver o perfil de produção de cada pesquisador, separando artigos, livros, congressos e outros tipos.

Comando:

```

SELECT
    p.nome,
    a.tipo,
    COUNT(*) AS quantidade
FROM PESQUISADORES p
JOIN PARTICIPACAO pa
    ON p.id_pesquisador = pa.id_pesquisador
JOIN ATIVIDADES a
    ON a.id_atividade = pa.id_atividade
GROUP BY
    p.nome,
    a.tipo
ORDER BY
    p.nome,
    quantidade DESC;

```

Como funciona: Os dois JOINS ligam a pessoa à sua participação e depois à atividade. O agrupamento por p.nome, a.tipo permite responder, por exemplo, quantos artigos e quantos congressos cada pesquisador possui.

8. Total de atividades por pesquisador

Por que usar: Descobrir quantas atividades diferentes estão associadas a cada pesquisador.

Comando:

```

SELECT
    p.nome,
    COUNT(DISTINCT pa.id_atividade) AS total_atividades
FROM PESQUISADORES p
JOIN PARTICIPACAO pa
    ON p.id_pesquisador = pa.id_pesquisador
GROUP BY
    p.id_pesquisador,
    p.nome
ORDER BY total_atividades DESC;

```

Como funciona: O COUNT(DISTINCT pa.id_atividade) evita que uma mesma atividade seja contada duas vezes para o mesmo pesquisador. O resultado permite observar o volume total de atividades associado a cada pessoa.

9. Colaborações entre pares por ano

Por que usar: Descobrir em quais anos dois pesquisadores específicos ou quaisquer pares de pesquisadores compartilharam atividades.

Comando:

```
SELECT
  p1.nome AS pesquisador_1,
  p2.nome AS pesquisador_2,
  a.ano,
  COUNT(*) AS atividades
FROM PARTICIPACAO pa1
JOIN PARTICIPACAO pa2
  ON pa1.id_atividade = pa2.id_atividade
  AND pa1.id_pesquisador < pa2.id_pesquisador
JOIN ATIVIDADES a
  ON a.id_atividade = pa1.id_atividade
JOIN PESQUISADORES p1
  ON p1.id_pesquisador = pa1.id_pesquisador
JOIN PESQUISADORES p2
  ON p2.id_pesquisador = pa2.id_pesquisador
GROUP BY
  p1.nome,
  p2.nome,
  a.ano
ORDER BY
  p1.nome,
  p2.nome,
  a.ano;
```

Como funciona: É uma extensão da consulta de coautoria. Aqui o agrupamento inclui a.ano, permitindo acompanhar temporalmente a colaboração de cada par.

Consultas focadas no pesquisador Luis Rodrigo Fernandes Baumann

10. Localizar o cadastro do pesquisador

Por que usar: Verificar se o pesquisador existe na tabela PESQUISADORES e consultar todos os seus dados cadastrais.

Comando:

```
SELECT *
FROM PESQUISADORES
WHERE nome = 'Luis Rodrigo Fernandes Baumann';
```

Como funciona: O WHERE restringe o resultado ao nome informado. O * retorna todas as colunas da tabela para esse pesquisador.

11. Listar atividades, papel e ordem de autoria

Por que usar: Ver todas as atividades ligadas ao pesquisador, incluindo seu papel e sua posição entre os autores.

Comando:

```
SELECT
  a.*,
  pa.papel,
  pa.ordem
FROM PESQUISADORES p
JOIN PARTICIPACAO pa
  ON pa.id_pesquisador = p.id_pesquisador
JOIN ATIVIDADES a
  ON a.id_atividade = pa.id_atividade
WHERE p.nome = 'Luis Rodrigo Fernandes Baumann'
ORDER BY a.ano DESC, a.titulo;
```

Como funciona: Primeiro a pessoa é ligada à tabela PARTICIPACAO; depois, cada participação é ligada à ATIVIDADES. O filtro seleciona apenas o pesquisador. pa.papel informa a função registrada e pa.ordem guarda a ordem de autoria quando disponível. A ordenação coloca os anos mais recentes primeiro.

12. Encontrar os coautores de Luis Rodrigo Fernandes Baumann

Por que usar: Identificar quem compartilhou atividades com esse pesquisador e quantas atividades eles têm em comum.

Comando:

```
SELECT
  p2.nome AS coautor,
  COUNT(DISTINCT pa1.id_atividade) AS atividades_em_comum
FROM PESQUISADORES p1
JOIN PARTICIPACAO pa1
  ON pa1.id_pesquisador = p1.id_pesquisador
JOIN PARTICIPACAO pa2
  ON pa2.id_atividade = pa1.id_atividade
  AND pa2.id_pesquisador <> pa1.id_pesquisador
JOIN PESQUISADORES p2
  ON p2.id_pesquisador = pa2.id_pesquisador
WHERE p1.nome = 'Luis Rodrigo Fernandes Baumann'
GROUP BY p2.id_pesquisador, p2.nome
ORDER BY atividades_em_comum DESC;
```

Como funciona: A consulta parte das atividades de Luis Rodrigo e procura outras pessoas presentes nessas mesmas atividades. A condição <> exclui o próprio pesquisador. COUNT(DISTINCT ...) conta quantas atividades diferentes os dois compartilharam.

13. Detalhar toda a produção de Luis Rodrigo Fernandes Baumann

Por que usar: Obter uma visão detalhada de cada atividade, incluindo título, ano, tipo, veículo, DOI, papel e ordem.

Comando:

```
SELECT
  p.nome AS pesquisador,
  a.id_atividade,
  a.titulo,
  a.ano,
  a.tipo,
  a.veiculo,
  a.doi,
  pa.papel,
  pa.ordem
FROM PESQUISADORES p
JOIN PARTICIPACAO pa
  ON pa.id_pesquisador = p.id_pesquisador
JOIN ATIVIDADES a
  ON a.id_atividade = pa.id_atividade
WHERE p.nome = 'Luis Rodrigo Fernandes Baumann'
ORDER BY a.ano DESC, a.tipo, a.titulo;
```

Como funciona: É uma consulta de auditoria e exploração. Em vez de usar a.*, ela escolhe explicitamente as colunas relevantes. Isso facilita exportar o resultado para análise, planilha ou relatório.

Padrões SQL que aparecem repetidamente

COUNT(*): Conta linhas dentro de cada conjunto de dados.

COUNT(DISTINCT ...): Conta valores únicos. É fundamental quando a mesma atividade ou pessoa pode aparecer várias vezes.

GROUP BY: Transforma registros em grupos para produzir contagens ou outras agregações.

ORDER BY: Define a ordem do resultado. DESC coloca os maiores valores primeiro.

JOIN: Combina informações de tabelas relacionadas.

JOIN PARTICIPACAO consigo mesma: É a técnica central para descobrir coautorias: dois registros apontam para a mesma atividade.

FILTER: Permite contar apenas as linhas que atendem a uma condição específica.

WHERE: Filtra os registros antes da agregação.

DISTINCT: Elimina duplicidades no cálculo ou no resultado.

<: Na consulta de pares, usar id_pesquisador < id_pesquisador impede A-B e B-A de aparecerem como dois pares.

Como essas consultas se conectam ao projeto

Em conjunto, essas consultas permitem sair do simples armazenamento dos currículos e começar a analisar a estrutura de colaboração acadêmica. A tabela ATIVIDADES representa os eventos/produtos; PARTICIPACAO funciona como a ponte entre pessoas e atividades; e os JOINS sobre essa ponte permitem reconstruir relações de coautoria. A partir daí, os resultados podem servir de base para análises como número de colaboradores, frequência de colaboração, evolução anual e construção de redes.

Observação: este documento explica exatamente os comandos SQL presentes no material fornecido. Ele não inclui comandos adicionais que não estavam no arquivo.